For a large number of vehicles and a complicated graph, this is a nontrivial combinatorial problem. The problem can be formulated as a discrete deterministic optimization problem, and addressed by approximate DP methods. The state at a given stage is the m-tuple of current positions of the vehicles together with the list of pending tasks, but the number of these states can be enormous (it increases exponentially with the number of nodes and the number of vehicles). Moreover the number of joint move choices by the vehicles also increases exponentially with the number of vehicles.

We are thus motivated to use a multiagent rollout approach. We define a base heuristic as follows: at a given stage and state (vehicle positions and pending tasks), it finds the closest pending task (in terms of number of moves needed to reach it) for each of the vehicles and moves each vehicle one step towards the corresponding closest pending task (this is a legitimate base heuristic: it assigns to each state a vehicle move for every vehicle).†

In the multiagent rollout algorithm, at a given stage and state, we take up each vehicle in the order $1, \ldots, n$, and we compare the Q-factors of the available moves to that vehicle while assuming that all the remaining moves will be made according to the base heuristic, and taking into account the moves that have been already made and the tasks that have already been performed; see the illustration of Fig. 2.9.3. In contrast to all-vehicles-at-once rollout, the one-vehicle-at-a-time rollout algorithm considers a polynomial (in m) number of moves and corresponding shortest path problems at each stage. In the example of Fig. 2.9.3, the one-vehicle-at-a-time rollout finds the optimal solution, while the base heuristic starting from the initial state does not.

The Cost Improvement Property

Generally, it is unclear how the two rollout policies (standard/all-agents-at-once and agent-by-agent) perform relative to each other in terms of attained cost.‡ On the other hand, both rollout policies perform no worse than the

† There is an alternative version of the base heuristic, which makes selections one-vehicle-at-a-time: at a given stage and state (vehicle positions and pending tasks), it finds the closest pending task (in terms of number of moves needed to reach it) for vehicle 1 and moves this vehicle one step towards this closest pending task. Then it finds the closest pending task for vehicle 2 (the pending status of the tasks, however, may have been affected by the move of vehicle 1) and moves this vehicle one step towards this closest pending task, and continues similarly for vehicles $3, \ldots, n$. There is a subtle difference between the two base heuristics: for example they may make different choices when vehicle 1 reaches a pending task in a single move, thereby changing the status of that task, and affecting the choice of the base heuristic for vehicle 2, etc.

‡ For an example where the standard rollout algorithm works better, consider a single-stage problem, where the objective is to minimize the first stage cost $g_0(u_0^1, \ldots, u_0^m)$. Let $\overline{u}_0 = (\overline{u}_0^1, \ldots, \overline{u}_0^m)$ be the control applied by the base policy, and assume that $\overline{u}_0$ is not optimal. Suppose that starting at $\overline{u}_0$, the cost cannot be improved by varying any single control component. Then the multiagent rollout algorithm stays at the suboptimal $\overline{u}_0$, while the standard rollout algorithm finds

base policy, since the performance of the base policy is identical for both the reformulated and the original problems. This cost improvement property can also be shown analytically as follows by induction, by modifying the standard rollout cost improvement proof; cf. Section 2.7.

Proposition 2.9.1: (Cost Improvement for Multiagent Rollout) The rollout policy $\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$ obtained by multiagent rollout satisfies

$$J_{k,\tilde{\pi}}(x_k) \leq J_{k,\pi}(x_k), \qquad \text{for all } x_k \text{ and } k, \tag{2.90}$$

where π is the base policy.

Proof: We will show the inequality (2.90) by induction, but for simplicity, we will give the proof for the case of just two agents, i.e., $m = 2$. Clearly the inequality holds for $k = N$, since $J_{N,\tilde{\pi}} = J_{N,\pi} = g_N$. Assuming that it holds for index $k+1$, we have for all x_k,

$$\begin{aligned}
J_{k,\tilde{\pi}}(x_k) &= E\Big\{g_k\big(x_k, \tilde{\mu}_k^1(x_k), \tilde{\mu}_k^2(x_k), w_k\big) \\
&\qquad\qquad + J_{k+1,\tilde{\pi}}\Big(f_k\big(x_k, \tilde{\mu}_k^1(x_k), \tilde{\mu}_k^2(x_k), w_k\big)\Big)\Big\} \\
&\leq E\Big\{g_k\big(x_k, \tilde{\mu}_k^1(x_k), \tilde{\mu}_k^2(x_k), w_k\big) \\
&\qquad\qquad + J_{k+1,\pi}\Big(f_k\big(x_k, \tilde{\mu}_k^1(x_k), \tilde{\mu}_k^2(x_k), w_k\big)\Big)\Big\} \\
&= \min_{u_k^2 \in U_k^2(x_k)} E\Big\{g_k(x_k, \tilde{\mu}_k^1(x_k), u_k^2, w_k) \\
&\qquad\qquad + J_{k+1,\pi}\Big(f_k\big(x_k, \tilde{\mu}_k^1(x_k), u_k^2, w_k\big)\Big)\Big\} \\
&\leq E\Big\{g_k\big(x_k, \tilde{\mu}_k^1(x_k), \mu_k^2(x_k), w_k\big) \\
&\qquad\qquad + J_{k+1,\pi}\Big(f_k\big(x_k, \tilde{\mu}_k^1(x_k), \mu_k^2(x_k), w_k\big)\Big)\Big\}
\end{aligned}$$

---

an optimal control. Thus, for one-stage problems, the standard rollout algorithm will perform no worse than the multiagent rollout algorithm.

The example just given is best seen within the framework of the classical coordinate descent method for minimizing a function of m components. This method can get stuck at a nonoptimal point in the absence of appropriate conditions on the cost function, such as differentiability and/or convexity. However, within our context of multistage rollout and possibly stochastic disturbances, it appears that the consequences of such a phenomenon may not be serious. In fact, one can construct multi-stage examples where multiagent rollout performs better than the standard rollout.

$$
\begin{aligned}
&= \min_{u_k^1 \in U_k^1(x_k)} E\Big\{ g_k(x_k, u_k^1, \mu_k^2(x_k), w_k) \\
&\qquad\qquad + J_{k+1,\pi}\Big(f_k\big(x_k, u_k^1, \mu_k^2(x_k), w_k\big)\Big)\Big\} \\
&\le E\Big\{ g_k\big(x_k, \mu_k^1(x_k), \mu_k^2(x_k), w_k\big) \\
&\qquad\qquad + J_{k+1,\pi}\Big(f_k\big(x_k, \mu_k^1(x_k), \mu_k^2(x_k), w_k\big)\Big)\Big\} \\
&= J_{k,\pi}(x_k),
\end{aligned}
$$

where:

(a) The first equality is the DP equation for the rollout policy $\tilde{\pi}$.

(b) The first inequality holds by the induction hypothesis.

(c) The second equality holds by the definition of the rollout algorithm as it pertains to agent 2.

(d) The third equality holds by the definition of the rollout algorithm as it pertains to agent 1.

(e) The fourth equality is the DP equation for the base policy π.

The induction proof of the cost improvement property (2.90) is thus complete for the case $m = 2$. The proof for an arbitrary number of agents m is entirely similar. **Q.E.D.**

Optimizing the Agent Order in Agent-by-Agent Rollout - Multiagent Parallelization

In the multiagent rollout algorithm described so far, the agents optimize the control components sequentially in a fixed order. It is possible to improve performance by trying to optimize at each stage k the order of the agents.

An efficient way to do this is to first optimize over all single agent Q-factors, by solving the m minimization problems that correspond to each of the agents $\ell = 1, \ldots, m$ being first in the multiagent rollout order. If ℓ_1 is the agent that produces the minimal Q-factor, we fix ℓ_1 to be the first agent in the multiagent rollout order. Then we optimize over all single agent Q-factors, by solving the $m - 1$ minimization problems that correspond to each of the agents $\ell \neq \ell_1$ being second in the multiagent rollout order. Let ℓ_2 be the agent that produces the minimal Q-factor, fix ℓ_2 to be the second agent in the multiagent rollout order, and continue in this manner. In the end, after

$$
m + (m-1) + \cdots + 1 = \frac{m(m+1)}{2} \tag{2.91}
$$

minimizations, we obtain an agent order $\ell_1, \ldots, \ell_m$ that produces a potentially much reduced Q-factor value, as well as the corresponding rollout control component selections.

The method just described likely produces substantially better performance, and eliminates the need for guessing a good agent order, but it increases the number of Q-factor calculations needed per stage roughly by a factor $(m+1)/2$. Still this is much better than the all-agents-at-once approach, which requires an exponential number of Q-factor calculations. Moreover, the Q-factor minimizations of the above process can be parallelized, so with m parallel processors, we can perform the number of $m(m+1)/2$ minimizations derived above in just m batches of parallel minimizations, which require about the same time as in the case where the agents are selected for Q-factor minimization in a fixed order. We finally note that our earlier cost improvement proof goes through again by induction, when the order of agent selection is variable at each stage k.

Multiagent Rollout Variants

The agent-by-agent rollout algorithm admits several variants. We describe briefly a few of these variants.

(a) We may use multiagent truncated rollout and terminal cost function approximation, as described earlier. Of course, in this case the cost improvement property need not hold.

(b) When the control constraint sets $U_k^\ell(x_k)$ are infinite, multiagent rollout still applies, based on the tradeoff between control and state space complexity, cf. Fig. 2.9.1. In particular, when the sets $U_k^\ell(x_k)$ are intervals of the real line, each agent's lookahead minimization problem can be performed with the aid of one-dimensional search methods.

(c) When the problem is deterministic there are additional possible variants of the multiagent rollout algorithm. In particular, for deterministic problems, we may use a more general base policy, i.e., a heuristic that is not defined by an underlying policy; cf. Section 2.3.1. In this case, if the sequential improvement assumption for the modified problem of Fig. 2.9.1 is not satisfied, then the cost improvement property may not hold. However, cost improvement may be restored by introducing fortification, as discussed in Section 2.3.2.

(d) The multiagent rollout algorithm can be simply modified to apply to infinite horizon problems. In this context, we may also consider policy iteration methods, which can be viewed as repeated rollout. These methods may involve agent-by-agent policy improvement, and value and policy approximations of intermediately generated policies (see the RL book [Ber19a], Section 5.7.3; also the paper [MLB25]). The paper [Ber21a] provides an analysis of methods involving exact policy evaluation and agent-by-agent policy improvement, and shows convergence to an agent-by-agent optimal policy, a form of locally optimal policy.

(e) The multiagent rollout algorithm can be simply modified to apply to deterministic continuous-time optimal control problems; cf. Section 2.6. The idea is again to simplify the minimization over $u(t)$ in the case where $u(t)$ consists of multiple components $u^1(t), \ldots, u^m(t)$.

(f) We can implement within the agent-by-agent rollout context the use of Q-factor differences. The motivation is similar: deal with the approximation errors that are inherent in the estimated cost of the base policy, $\tilde{J}_{k+1,\pi}\big(f_k(x_k, u_k)\big)$, and may overwhelm the current stage cost term $g_k(x_k, u_k)$. This may seriously degrade the quality of the rollout policy; see also the discussion of advantage updating and differential training in Chapter 3.

Constrained Multiagent Rollout

Let us consider a special structure of the control space, where the control u_k consists of m components, $u_k = (u_k^1, \ldots, u_k^m)$, each belonging to a corresponding set $U_k^\ell(x_k)$, $\ell = 1, \ldots, m$. Thus the control space at stage k is the Cartesian product

$$U_k(x_k) = U_k^1(x_k) \times \cdots \times U_k^m(x_k).$$

We refer to this as the *multiagent case*, motivated by the special case where each component u_k^ℓ is chosen by a separate agent ℓ at stage k.

Similar to the unconstrained case, we can introduce a modified but equivalent problem, involving one-at-a-time agent control selection. In particular, at the generic state x_k, we break down the control u_k into the sequence of the m controls $u_k^1, u_k^2, \ldots, u_k^m$, and between x_k and the next state $x_{k+1} = f_k(x_k, u_k)$, we introduce artificial intermediate "states"

$$(x_k, u_k^1), (x_k, u_k^1, u_k^2), \ldots, (x_k, u_k^1, \ldots, u_k^{m-1}),$$

and corresponding transitions. The choice of the last control component u_k^m at "state" $(x_k, u_k^1, \ldots, u_k^{m-1})$ marks the transition at cost $g_k(x_k, u_k)$ to the next state $x_{k+1} = f_k(x_k, u_k)$ according to the system equation. It is evident that this reformulated problem is equivalent to the original, since any control choice that is possible in one problem is also possible in the other problem, with the same cost.

By working with the reformulated problem, we can consider a rollout algorithm requires a sequence of m minimizations per stage, one over each of the control components $u_k^1, \ldots, u_k^m$, with the past controls already determined by the rollout algorithm, and the future controls determined by running the base heuristic. Assuming a maximum of n elements in the control component spaces $U_k^\ell(x_k)$, $\ell = 1, \ldots, m$, the computation required for the m single control component minimizations is of order $O(nm)$ per stage. By contrast the standard rollout minimization (2.47) involves the computation of as many as n^m terms $G\big(T_k(\tilde{y}_k, u_k)\big)$ per stage.

2.9.1 Asynchronous and Autonomous Multiagent Rollout

In this section we consider multiagent rollout algorithms that are distributed and asynchronous in the sense that the agents may compute their rollout controls in parallel rather than in sequence, aiming at computational speedup. An example of such an algorithm is obtained when at a given stage, agent ℓ computes the rollout control $\tilde{u}_k^\ell$ before knowing the rollout controls of some of the agents $1, \ldots, \ell - 1$, and uses the controls $\mu_k^1(x_k), \ldots, \mu_k^{\ell-1}(x_k)$ of the base policy in their place.

This algorithm often works well and it may be worth trying on a given problem. However, it does not possess the cost improvement property, and may not work well at all for some problems and specially selected initial states. In fact we can construct a simple example with a single state, two agents, and two controls per agent, where the second agent does not take into account the control applied by the first agent, and as a result the rollout policy performs worse than the base policy for some initial states.

Example 2.9.3 (Cost Deterioration in the Absence of Adequate Agent Coordination)

Consider a problem with two agents ($m = 2$) and a single state. Thus the state does not change and the costs of different stages are decoupled (the problem is essentially static). Each of the two agents has two controls: $u_k^1 \in \{0, 1\}$ and $u_k^2 \in \{0, 1\}$. The cost per stage g_k is equal to 0 if $u_k^1 \neq u_k^2$, is equal to 1 if $u_k^1 = u_k^2 = 0$, and is equal to 2 if $u_k^1 = u_k^2 = 1$. Suppose that the base policy applies $u_k^1 = u_k^2 = 0$. Then it can be seen that when executing rollout, the first agent applies $u_k^1 = 1$, and in the absence of knowledge of this choice, the second agent also applies $u_k^2 = 1$ (thinking that the first agent will use the base policy control $u_k^1 = 0$). Thus the cost of the rollout policy is 2 per stage, while the cost of the base policy is 1 per stage. By contrast the rollout algorithm that takes into account the first agent's control when selecting the second agent's control applies $u_k^1 = 1$ and $u_k^2 = 0$, thus resulting in a rollout policy with the optimal cost of 0 per stage.

The difficulty here is inadequate coordination between the two agents. In particular, each agent uses rollout to compute the local control, thinking that the other will use the base policy control. If instead the two agents coordinated their control choices, they would have applied an optimal policy.

The simplicity of the preceding example raises serious questions as to whether the cost improvement property (2.90) can be easily maintained by a distributed rollout algorithm where the agents do not know the controls applied by the preceding agents in the given order of local control selection, and use instead the controls of the base policy. One may speculate that if the agents are naturally "weakly coupled" in the sense that their choice of control has little impact on the desirability of various controls of other

agents, then a more flexible inter-agent communication pattern may be sufficient for cost improvement.†

An important question is to clarify the extent to which agent coordination is essential. In what follows in this section, we will discuss a distributed asynchronous multiagent rollout scheme, which is based on the use of a signaling policy that provides estimates of coordinating information once the current state is known.

Autonomous Multiagent Rollout - Signaling Policies

An interesting possibility for autonomous control selection by the agents is to use a distributed rollout algorithm, which is augmented by a precomputed signaling policy that embodies agent coordination.‡ The idea is to assume that the agents do not communicate their computed rollout control components to the subsequent agents in the given order of local control selection. Instead, *once the agents know the state, they use precomputed (or easily computed) approximations to the control components of the preceding agents*, and compute their own control components in parallel and asynchronously. We call this algorithm *autonomous multiagent rollout*. While this type of algorithm involves a form of redundant computation, it allows for additional speedup through parallelization.

The algorithm at the kth stage uses a base policy $\mu_k = \{\mu_k^1, \ldots, \mu_k^{m-1}\}$, but also uses *a second policy* $\widehat{\mu}_k = \{\widehat{\mu}_k^1, \ldots, \widehat{\mu}_k^{m-1}\}$, called the *signaling policy*, which is computed off-line, is known to all the agents for on-line use, and is designed to play an agent coordination role. Intuitively, $\widehat{\mu}_k^\ell(x_k)$ *provides an intelligent "guess" about what agent ℓ will do at state x_k*. This is used in turn by all other agents $i \neq \ell$ to compute asynchronously their own rollout control components on-line.

More precisely, the autonomous multiagent rollout algorithm uses the base and signaling policies to generate a rollout policy $\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$

† In particular, one may divide the agents in "coupled" groups, and require coordination of control selection only within each group, while the computation of different groups may proceed in parallel. Note that the "coupled" group formations may change over time, depending on the current state. For example, in applications where the agents' locations are distributed within some geographical area, it may make sense to form agent groups on the basis of geographic proximity, i.e., one may require that agents that are geographically near each other (and hence are more coupled) coordinate their control selections, while agents that are geographically far apart (and hence are less coupled) forego any coordination.

‡ The general idea of coordination by sharing information about the agents' policies arises also in other multiagent algorithmic contexts, including some that involve forms of policy gradient methods and Q-learning; see the surveys of the relevant research cited earlier. The survey by Matignon, Laurent, and Le Fort-Piat [MLL12] focuses on coordination problems from an RL point of view.

as follows. At stage k and state x_k, $\tilde{\mu}_k(x_k) = \big(\tilde{\mu}_k^1(x_k), \ldots, \tilde{\mu}_k^m(x_k)\big)$, is obtained according to

$$\tilde{\mu}_k^1(x_k) \in \arg \min_{u_k^1 \in U_k^1(x_k)} E\Big\{g_k\big(x_k, u_k^1, \mu_k^2(x_k), \ldots, \mu_k^m(x_k), w_k\big) + J_{k+1,\pi}\Big(f_k\big(x_k, u_k^1, \mu_k^2(x_k), \ldots, \mu_k^m(x_k), w_k\big)\Big)\Big\},$$

$$\tilde{\mu}_k^2(x_k) \in \arg \min_{u_k^2 \in U_k^2(x_k)} E\Big\{g_k\big(x_k, \widehat{\mu}_k^1(x_k), u_k^2, \ldots, \mu_k^m(x_k), w_k\big) + J_{k+1,\pi}\Big(f_k\big(x_k, \widehat{\mu}_k^1(x_k), u_k^2, \ldots, \mu_k^m(x_k), w_k\big)\Big)\Big\},$$

$$\ldots \qquad \ldots \qquad \ldots$$

$$\tilde{\mu}_k^m(x_k) \in \arg \min_{u_k^m \in U_k^m(x_k)} E\Big\{g_k\big(x_k, \widehat{\mu}_k^1(x_k), \ldots, \widehat{\mu}_k^{m-1}(x_k), u_k^m, w_k\big) + J_{k+1,\pi}\Big(f_k\big(x_k, \widehat{\mu}_k^1(x_k), \ldots, \widehat{\mu}_k^{m-1}(x_k), u_k^m, w_k\big)\Big)\Big\}. \tag{2.92}$$

Note that the preceding computation of the controls $\tilde{\mu}_k^1(x_k), \ldots, \tilde{\mu}_k^m(x_k)$ can be done asynchronously and in parallel, and without direct agent coordination, since the signaling policy values $\widehat{\mu}_k^1(x_k), \ldots, \widehat{\mu}_k^{m-1}(x_k)$ are precomputed and are known to all the agents.

The simplest choice is to *use as signaling policy $\widehat{\mu}$ the base policy μ*. However, this choice does not guarantee policy improvement as evidenced by Example 2.9.3. In fact performance deterioration with this choice is not uncommon, and can be observed in more complicated examples, including the following.

Example 2.9.4 (Spiders and Flies - Use of the Base Policy for Signaling)

Consider the problem of Example 2.9.1, which involves two spiders and two flies on a line, and the base policy μ that moves a spider towards the closest surviving fly (and in case where a spider starts at the midpoint between the two flies, moves the spider to the right). Assume that we use as signaling policy $\widehat{\mu}$ the base policy μ. It can then be verified that if the spiders start from different positions, the rollout policy will be optimal (will move the spiders in opposite directions). If, however, the spiders start *from the same position*, a completely symmetric situation is created, whereby the rollout controls move both flies in the direction of the fly *furthest away* from the spiders' position (or to the left in the case where the spiders start at the midpoint between the two flies). Thus, the flies end up oscillating around the middle of the interval between the flies and never catch the flies!

The preceding example is representative of a broad class of counterexamples that involve multiple identical agents. If the agents start at